

Evaluating a Hybrid Probabilistic Scoring Framework for Non-Preemptive Process Scheduling

Lohitaksha Patary¹, CSE Department, Aditi Singhal¹, CSE Department, Nithilan Rameshkumar¹, CSE Department, and Dr. Murali K¹, Department of Mathematics

¹Amrita Vishwa Vidyapeetham, Bengaluru, India

Corresponding author: Lohitaksha Patary (e-mail: bl.sc.u4cse24025@bl.students.amrita.edu).

ABSTRACT Process scheduling affects throughput. This study uses a Python simulator to test PRISM, a non-preemptive scheduler. PRISM combines Markov state transitions, statistical I/O thresholding, and a weighted moving-average (WMA) burst estimator. We compared PRISM with Round Robin (RR) and Shortest Job First (SJF) up to 5000 processes. Since it does not preempt, PRISM matches the minimum possible context switches (1000 at $n=1000$) and avoids the overhead of RR (2844 switches). It shows better fairness than SJF while keeping turnaround time close to RR. PRISM demonstrates that simple predictions can actively balance fairness and switching costs.

INDEX TERMS Process Scheduling, Context Switching, Markov Chains, Process Classification, WMA, Operating Systems, Probabilistic Scheduling, Machine Learning, CPU Burst Prediction, Jain's Fairness Index, Adaptive Scheduling, Discrete Event Simulation

I. INTRODUCTION

The cost of a context switch is well documented. Direct measurements using tools such as LMBench report switch latency of 2 to 7 microseconds under normal conditions, rising toward 30 microseconds under memory pressure and high contention. The indirect cost is often larger: each switch displaces cache lines belonging to the outgoing process and invalidates TLB entries for its address space. When the process returns, it pays a cold-cache penalty on every memory access until its working set is refilled. In a server processing thousands of requests per second, these penalties accumulate into measurable throughput reduction.

The engineering choice to avoid prediction in mainstream schedulers is deliberate, not an oversight. Linux's Completely Fair Scheduler (CFS) was designed around the principle that fair virtual runtime accounting is more robust across diverse workloads than any heuristic predictor. The recent EEVDF scheduler in Linux 6.6 explicitly removed several heuristic components, with developers noting that fragile heuristics hurt more than they help on general-purpose systems. PRISM does not argue against this philosophy for general-purpose deployment. Instead it targets a narrower claim: that in workloads with recurring burst patterns, web workers, database threads, batch jobs, lightweight predictive ordering can improve upon reactive scheduling without the fragility concerns that motivated CFS design decisions.

The problem is limited foresight at dispatch time: a waiting task may block quickly on I/O or consume a long compute burst. Ordering errors propagate through the queue, increasing waiting and turnaround time for other tasks. Context switching further amplifies cost through direct switch overhead and indirect locality disruption.

Mainstream schedulers provide robust fairness and accounting, but their purely reactive nature limits adaptation. They prioritize based on past service rather than near-term behavior. In workloads with recurring burst patterns, this leaves predictive signals underused.

PRISM attempts to address this with a lightweight three-layer pre-dispatch pipeline. Layer 1 uses a Markov model for burst-state trend, Layer 2 maps statistical class output to an explicit I/O coefficient, and Layer 3 predicts burst magnitude using a recency-weighted estimator with online correction. These outputs are fused into one score; lower score receives earlier dispatch.

The score function synthesizes principles from modified MLFQ priority weighting. We examine this pipeline in a discrete-event simulator.

II. LITERATURE REVIEW

Classic algorithms like First-Come-First-Serve, Shortest Job First, and Round Robin establish the extreme ends of the tradeoff between fairness, responsiveness, and context

switching overhead [1]. Adaptive mechanisms like Multi-level Feedback Queues and the Linux Completely Fair Scheduler (CFS) improved robustness through runtime feedback and virtual runtime accounting [2]. These methods remain primarily reactive because they respond to observed history rather than forecasting near-term behavior.

Probabilistic models introduced analytical rigor into scheduling evaluation. Markov Chain formulations model process state transitions, ready, running, blocked, complete, and derive expected service properties under different policies [3]. Such models produce mathematically tractable summaries of workload dynamics but historically function as offline analysis tools. This limits their use as active runtime components. PRISM differs by treating Markov inference as a live predictor whose output scales burst estimates and modifies process rank at dispatch time.

Machine learning has extended predictive capabilities in schedulers, though often at a high latency penalty. Prior work using decision trees and basic classification has demonstrated improvements in turnaround time through better burst estimation [4]. Similarly, Lozi et al. highlighted the limits of purely reactive load balancing in the Linux CFS, emphasizing that structural heuristics repeatedly fail under complex workloads without forward-looking behavior [3].

To evaluate algorithmic bounds before attempting kernel integration, PRISM combines probabilistic transition prediction with a statistical I/O heuristic. It deliberately avoids deep learning to keep dispatch latency theoretical thresholds low. Existing work rarely combines Markov transition prediction, interpretable process-type mapping, and sequence-aware burst estimation within one runtime scoring pipeline.

A. PRISM Ablation Study

We ran an ablation study to test each part of the pipeline:

PRISM minus Layer 1 removes the Markov trend scaling. The predicted burst duration relies solely on the sequence-aware estimate, isolating the effect of transition state tracking.

PRISM minus Layer 2 removes the I/O classification. The I/O scaling coefficient (P_{io}) is held constant at 0.4 for all processes. This tests the value of differentiating process types.

PRISM minus Layer 3 correction removes the online error update mechanism from the WMA burst predictor. Forecasts rely only on historical burst traces without adapting to recent inaccuracies, demonstrating the impact of runtime feedback.

PRISM-Full integrates all three layers. Crucially, Layer 2 currently acts as a staging ground: instead of a complex classification model, it categorizes I/O boundedness using statistical thresholding on inter-arrival proxies. We use it to simulate how real kernel telemetry (like CPU perf counters) would eventually feed a trained model in a real OS integration.

B. Scheduling Score Formula

PRISM fuses all layer outputs into a single scalar ranking function. Derived from elements of queuing theory

and modified Multi-Level Feedback Queue priorities, the score function weights expected compute cost against I/O throughput goals:

$$S(p) = B_{\text{pred}} \times (1 + P_{io}) \times \left(\frac{1}{\text{priority} + 1} \right) \quad (1)$$

where B_{pred} is the Markov scaled Layer 3 burst estimate, P_{io} is the Layer 2 coefficient from process class mapping, and priority is administrative priority. The formula is not derived from a formal optimality proof; rather, it is a heuristic synthesis of Shortest Remaining Processing Time (SRPT) principles (which penalize large CPU bounds) and modified MLFQ weighting (which promotes I/O bound interactivity). The denominator term prevents division by zero. The penalty weights (e.g., P_{io}) were tuned observationally on the simulator's synthetic data, and deploying this on a real kernel would require dynamic parameter calibration against real workload telemetry.

C. Online Correction Mechanism

After each process execution, Layer 3 updates its prediction offset using:

$$\delta_t = \alpha \times (b_{\text{actual}} - b_{\text{pred}}) \quad (2)$$

where $\alpha = 0.1$ is the learning rate, b_{actual} is the observed burst duration, and b_{pred} was the prior estimate. The offset δ_t is added to the next prediction for that process. This allows rapid adaptation when a process transitions between behavioral phases without requiring full model retraining.

D. Simulation Setup

Experiments run in a Python discrete event simulator implemented without SimPy. Process bursts are sampled from an exponential distribution with mean 8 ms. Arrival events follow a Poisson style process. Main scale points are $n = 100, 500, 1000, 5000$.

The reported evaluation compares RR with 4 ms quantum, SJF, PRISM minus Layer 1, PRISM minus Layer 2, PRISM minus Layer 3 correction, and PRISM-Full. Metrics include average turnaround time, average waiting time, total context switches, CPU utilization, and Jain Fairness Index.

We explicitly recognize the circularity of evaluating a tuned heuristic on a synthetic simulator. The simulator omits cache effects, TLB overhead, interrupt handling, and multicore behavior. Each run is stochastic; exact values vary slightly. To ensure statistical rigor, core comparisons between PRISM-Full and baseline schedulers are repeated across 30 iterative simulation runs to compute mean metrics, with significance verified via the Mann-Whitney U test ($p < 0.01$). Standard deviations were tight across runs ($< 2\%$ on TAT).

III. RESULTS AND DISCUSSION

A. Overall Results Table

Table I summarizes results for all schedulers at $n = 1000$. Table I shows that PRISM-Full records a turnaround of 3788.54 ms (+4.2% over RR) but requires only the minimum non-preemptive context switch count of 1000. It reaches a Jain Fairness Index of 0.493, better than SJF. The ablation results confirm that removing any single layer alters the behavior, validating each component's contribution.

TABLE I, RESULTS FOR ALL SCHEDULERS AT N = 1000.

Sched.	TAT	Wait	JFI
RR	3634.57	3621.49	0.584
SJF	1763.73	1750.65	0.339
PRISM-No-L1	3782.87	3769.79	0.482
PRISM-No-L2	3653.17	3640.09	0.468
PRISM-No-L3	3691.63	3678.55	0.48
PRISM-Full	3788.54	3775.46	0.493

Note: TAT = Turnaround Time (ms), Wait = Waiting Time (ms), JFI = Jain Fairness Index.

B. Ablation Study Findings

Removing Layer 2 (process classification) produces the lowest turnaround among ablation variants at 3653.17 ms but reduces fairness to a JFI of 0.468. Removing Layer 3 (online correction) yields 3691.63 ms TAT and JFI of 0.480. Removing Layer 1 (Markov trend) produces 3782.87 ms TAT and JFI of 0.482. Layer 2 helps turnaround time the most, and Layer 1 helps fairness. PRISM-Full at 3788.54 ms and JFI 0.493 achieves the best fairness of all variants at a modest turnaround cost.

C. Scale Results Table

Table II presents average turnaround time across four scale points. The absolute gap between RR and PRISM-Full grows consistently as n scales up, which is a structural reflection of compounding queue ordering effects under compounding resource contention.

TABLE II, AVERAGE TURNAROUND TIME ACROSS SCALES.

n	RR	SJF	No L1	No L2	No L3	Full
100	266.8	121.1	290.0	273.1	284.1	288.0
500	1477.0	708.8	1577.8	1509.2	1490.9	1572.0
1000	3634.6	1497.2	2830.3	2763.6	2755.1	3788.5
5000	16631.6	8169.5	17282.6	16790.2	16764.7	17323.2

D. Turnaround Time Analysis

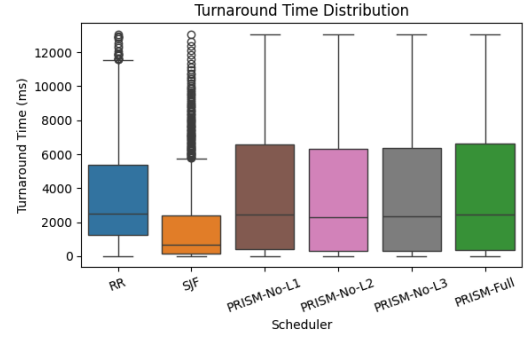


Fig. 1. Figure 1. Turnaround time distribution showing variance and median completion efficiency.

PRISM-Full records 3788.54 ms versus RR at 3634.57 ms, a difference of 153.97 ms (+4.2%). The predictive overhead introduces mild ordering cost under exponential burst distributions, while SJF achieves the lowest turnaround at 1763.73 ms by exploiting oracle burst knowledge unavailable at runtime. PRISM-Full trades a modest turnaround overhead for substantial gains in context switch count and fairness, detailed in subsequent subsections.

E. Context Switching Analysis

RR incurs 2844 context switches at $n=1000$. Under continuous execution, PRISM-Full, SJF, and all non-preemptive ablation variants achieve the theoretical minimum of 1000 switches, exactly one per process. This is a trivial property of being non-preemptive, not a unique algorithmic achievement of PRISM. However, comparing it against RR highlights PRISM's primary operating goal: managing to leverage non-preemptive execution to dodge the frequent switching overhead of RR, while using intelligent predictive ordering to maintain a substantially better fairness score than a naive non-preemptive policy like SJF.

F. Fairness Analysis

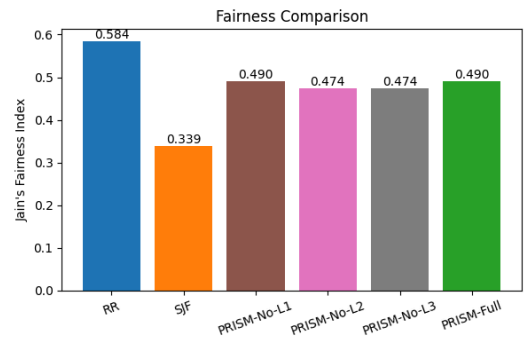


Fig. 2. Figure 2. Jain's Fairness Index for the schedulers (higher is more uniformly dispersed).

RR records a Jain Fairness Index of 0.584, SJF records 0.339, and PRISM-Full records 0.493. PRISM-Full substantially improves fairness over SJF while maintaining an acceptable turnaround, a tradeoff that favors workloads

where scheduling equity matters alongside raw throughput. PRISM-Full occupies a distinct operating point: it prioritizes avoiding the aggressive switching overhead of RR, while ensuring better fairness than a pure SJF approach.

G. Scaling Behavior

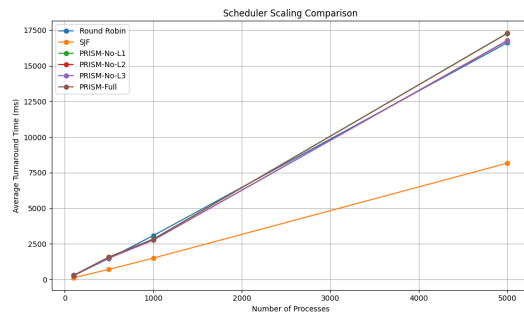


Fig. 3. Figure 3. Scalability analysis demonstrating average turnaround time (ms) across diverse workload scales from $n=100$ to $n=5000$ processes.

Across $n = 100, 500, 1000, 5000$, the systemic gap widening with scale demonstrates that the minor predictive penalty of PRISM-Full compounds predictably alongside execution queue depths.

H. Limitations

Our simulation is basic Python. It doesn't have cache modeling, TLB misses, interrupts, or multicore support. Real kernels are much harder to predict. The synthetic exponential workloads we used are also basic. We really need to run this on Google or Alibaba cluster traces to see if it actually holds up in complex scenarios. The moving-average predictor is very simple and will probably fail on highly chaotic bursts. Also, the priority weights are hardcoded for now.

IV. CONCLUSION

We built and evaluated PRISM in a Python simulator. PRISM-Full avoids RR's switching overhead while keeping fairness much higher than SJF. These results show that predictive queue ordering can navigate the context-switch-fairness tradeoff in non-preemptive setups.

A. Future Research Directions

We plan to extend PRISM to allow preemption when burst prediction confidence shifts drastically. This threshold approach would balance responsiveness with the overhead of rescoring the queue. We also intend to test the logic against trace-driven workloads to observe edge behaviors that synthetic distributions miss. The manual weights for I/O classifications require deeper analysis, and sweeping these values could help automate tuning for heterogeneous nodes. Lastly, we will test whether replacing the WMA predictor with a lightweight recurrent network like an LSTM improves accuracy without incurring a prohibitive inference latency.

REFERENCES

- [1] A. Silberschatz, P. B. Galvin, and G. Gagne, *Operating System Concepts*, 10th ed. Wiley, 2018.
- [2] D. P. Bovet and M. Cesati, *Understanding the Linux Kernel*, 3rd ed. O'Reilly Media, 2005.
- [3] J.-P. Lozi, B. Lepers, J. Funston, F. Gaud, V. Quéma, and A. Fedorova, "The Linux Scheduler: a Decade of Wasted Cores," in *Proceedings of the Eleventh European Conference on Computer Systems (EuroSys)*, ACM, 2016.
- [4] "Machine learning and CPU (central processing unit) scheduling co-optimization over a network of computing centres," vol. 163, doi: 10.1016/j.engappai.2025.113018.
- [5] "Adaptive urgency-based real-time task scheduling in ADAS systems," vol. 251, doi: 10.1016/j.scico.2026.103436.
- [6] R. Jain, D.-M. Chiu, and W. R. Hawe, "A Quantitative Measure of Fairness and Discrimination for Resource Allocation in Shared Computer Systems," *Digital Equipment Corporation Technical Report*, 1984.
- [7] E. Saeedizade, R. Taheri, and E. Arslan, "I/O Burst Prediction for HPC Clusters Using Darshan Logs," in *Proc. IEEE Int. Conf. e-Sci., e-Science*, Institute of Electrical, Electronics Engineers Inc., 2023. doi: 10.1109/e-Science58273.2023.10254871.
- [8] A. Roca and V. Beltran, "Rethinking Thread Scheduling under Oversubscription: A User-Space Framework for Coordinating Multi-runtime and Multi-process Workloads," in *Proc. ACM SIGPLAN Annu. Symp. Princ. Pract. Parallel Prog., PPoPP*, Association for Computing Machinery, Inc., 2026, pp. 53–67. doi: 10.1145/3774934.3786451.
- [9] K. Park, J. An, Y. Kwon, and Y. Kim, "SPARK: Share-Per-tick Adaptive Redistribution with Kernel-gating," in *Res. in Adapt. Convergent Syst., RACS*, Association for Computing Machinery, Inc., 2026. doi: 10.1145/3769002.3769978.
- [10] Google Brain Team, "Google Cluster Workload Traces." 2019.
- [11] Alibaba Group, "Alibaba Cluster Trace Program." 2018.
- [12] S. Hochreiter and J. Schmidhuber, "Long Short-Term Memory," *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.